Experiment No -04 (Group B)

Title:-

Implementing a real-time Undo/Redo System using Stack (Push and Pop operations) in Python

Problem statement:-

Implementing a real-time undo/redo system for a text editing application using a Stack data structure. The system should support the following operations:

•Make a Change: A new change to the document is made.

•Undo Action: Revert the most recent change and store it for potential redo.

•Redo Action: Reapply the most recently undone action.

•Display Document State: Show the current state of the document after undoing or redoing an action

Pre-requisite:-

- Basics of Python programming.
- Understanding of the Stack data structure and its LIFO (Last In, First Out) principle.
- Python list operations (append(), pop()).
- Functions and loops in Python.

Objective:-

To understand the implementation of the **Stack** data structure.

To implement a real-time **Undo/Redo** system using two stacks.

To perform operations such as **Make Change**, **Undo**, **Redo**, and **Display Current Document State**.

To understand how stacks are used in real-world applications like text editors.

Software and Hardware requirements:-

1. **Operating system:** Linux- Ubuntu 16.04 to 17.10, or Windows 7 to 10,
2. **RAM-** 2GB RAM (4GB preferable)
3. IDE :- Python 3
4. **Hardware**: Standard PC with keyboard input

Outcome:-

After completing this experiment, students will be able to:

Explain the working principle of the Stack (LIFO).

Implement an efficient Undo/Redo mechanism using two stacks.

Perform document editing operations while maintaining previous document states.

Understand the practical application of stacks in software development and text editing systems.

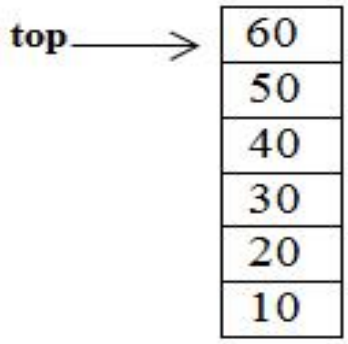
Theory:-

Basic concept of Stack:-

Definition:-

A stack is an ordered list in which all the insertion and deletion are made at one end called as top.

If we have to make stack elements of 10, 20, 30, 40, 50, 60 then 10 will be the bottommost element and 60 will be the topmost element in the stack. As stack is shown in figure



e.g. Stack of coin.

The coins can be arranged one on other when we add a new coin it is always placed on previous coin while removing the coin the recently placed coin can be removed.

Stack Abstract Data Type:-

Abstract Data Type Stack

{Instance :-

Stack is a collection of elements in which insertion n& deletion of elements is done by one end called as top.

Preconditions:-

1.stfull( ):-

This condition indicates whether the Stack is full or not. If Stack is full then we cannot insert the elements in the Stack.

2. stempty( ):-

This condition indicates whether the Stack is empty or not. If Stack is empty then we cannot pop or remove any element from the Stack.

Operations:-

1.Push :-

By this operation one can push elements onto the stack. Before performing the push we must check stfull( ) condition.

2. Pop:-

By this operation one can remove elements from the stack. Before performing the pop we must check stempty( ) condition.

}

In a text editor, users often need to revert changes (undo) or reapply changes (redo). This can be efficiently implemented using two Stacks:

Undo Stack stores all previous states.

Redo Stack stores the states that have been undone.

When a new change is made:

Push current state onto the Undo Stack.

Clear the Redo Stack (since redo history becomes invalid).

When performing Undo:

Pop the last state from the Undo Stack and push current state onto Redo Stack.

When performing Redo:

Pop from the Redo Stack and push the current state onto the Undo Stack.

Algorithm:-

Step 1: Start.

Step 2: Create a TextEditor object and initialize:

- text = ""
- undo = []
- redo = []

Step 3: Display the following menu repeatedly:

1. Add Text
2. Undo
3. Redo
4. Display
5. Exit

Step 4: Accept the user's choice.

If Choice = 1: Add Text

1. Accept new text from the user.
2. Store the current text in the undo stack.
3. Append the new text to the current text.
4. Clear the redo stack because a new change has been made.
5. Display the updated text.
6. Go back to the main menu.

If Choice = 2: Undo

1. Check whether the undo stack is empty.
2. If it is not empty:
    - Push the current text into the redo stack.
    - Pop the last text from the undo stack.
    - Make the popped text the current text.
    - Display **"Undo successful."**

3. Otherwise, display **"Nothing to undo."**
4. Display the current text.
5. Go back to the main menu.

If Choice = 3: Redo

1. Check whether the redo stack is empty.
2. If it is not empty:
    - Push the current text into the undo stack.
    - Pop the last text from the redo stack.
    - Make the popped text the current text.
    - Display **"Redo successful."**
3. Otherwise, display **"Nothing to redo."**
4. Display the current text.
5. Go back to the main menu.

If Choice = 4: Display

1. Display the current text.
2. Go back to the main menu.

If Choice = 5: Exit

1. Display **"Program ended."**
2. Stop the program.

If Choice is Invalid

1. Display **"Invalid choice."**
2. Go back to the main menu.

Step 5: Repeat Steps 3–4 until the user selects **Exit**.

Step 6: Stop.

Conclusion:-

The experiment successfully demonstrates the implementation of a **real-time Undo/Redo system** using the **Stack** data structure.

Viva Questions:-

1. What is a Stack?
2. What does LIFO stand for?
3. Why is a Stack used in an Undo/Redo system?
4. How many stacks are required for an Undo/Redo system?
5. What happens when a new change is made?
6. What happens during an Undo operation?
7. What happens during a Redo operation?
8. Which Stack operation is used to insert an element?
9. Which Stack operation is used to remove an element?
10. What is the Peek operation?
11. What is the advantage of using two stacks instead of one?
12. What are the applications of an Undo/Redo system?